Universidad Mariano Gálvez de Guatemala

Facultad de Ingeniería en Sistemas

Proyecto del curso



ER-AFD

Convertidor de expresiones regulares a autómatas finitos deterministas

Documento 01
Reporte técnico

Integrantes

Carlos Abraham Méndez Asencio - Carné 94902417370

Maycol Alexander Ortiz Flores - Carné 9490-24-27213

Objetivo y organización del programa

El proyecto ER-AFD se desarrolló para reconocer cadenas completas mediante expresiones regulares y autómatas finitos. El programa no utiliza un motor de expresiones regulares para decidir si una cadena pertenece o no al lenguaje. Primero interpreta la expresión, construye un autómata finito no determinista (AFND) con el procedimiento de Thompson y luego obtiene un autómata finito determinista (AFD) aplicando la construcción de subconjuntos.

El resultado se determina al recorrer la matriz de transiciones del AFD, carácter por carácter. La herramienta también muestra los estados, alfabetos, transiciones y recorrido de la cadena. Como función adicional se incorporó er-lex, que permite trabajar con varias reglas y reconocer tokens en un texto o archivo.

Módulos principales

Archivo	Función en el proyecto
parser.py	Valida la expresión regular y construye el árbol de sintaxis.
automata.py	Construye el AFND mediante Thompson, calcula cierres épsilon y convierte a AFD.
cli.py	Interpreta argumentos, lee entradas ASCII y devuelve los códigos 1, 0 o 2.
lexer.py	Reconoce tokens a partir de las matrices de varios AFD.
tests/ y tools/	Contienen pruebas, construcción, verificadores y evidencias reproducibles.

Entorno comprobado

Según los registros del proyecto, la verificación local se realizó el 2 de noviembre de 2026 en Ubuntu Desktop 26.04.1 LTS de 64 bits, con Python 3.14.4 y GNU Make 4.4.1, mediante un usuario regular. La publicación completa del repositorio y los resultados de una ejecución de GitHub Actions se documentan por separado; no se presentan como tareas ya terminadas.

Sintaxis y representación

El analizador sintáctico utiliza descenso recursivo. La gramática separa la unión, la concatenación, la repetición y los átomos de una expresión regular. Esta organización permite aplicar correctamente la precedencia de las operaciones y evita construir árboles innecesariamente profundos cuando hay varias uniones o concatenaciones.

```
union  := concat ("|" concat)*
concat := repeat repeat*
repeat := atom "*"*
atom   := literal | "." | "ε" | clase
          | escape | "(" union ")"
```

Primero se reconoce un átomo o grupo, después la cerradura de Kleene, luego la concatenación y, por último, la unión. Por ejemplo, ab* significa una a seguida de cero o más b; (ab)* representa cero o más repeticiones del bloque completo ab.

Familia	Elementos admitidos
Operaciones básicas	Concatenación, unión |, cerradura *, paréntesis, punto y ε.
Conjuntos	[abc], conjuntos negativos como [^abc] y rangos inclusivos como [a-z].
Escapes	Caracteres literales con diagonal invertida, incluida la propia diagonal.
Secuencias especiales	\N, \A, \a, \z y códigos hexadecimales \hXX entre 00 y 7F.

Universo de caracteres

El programa utiliza el conjunto ASCII completo de 128 caracteres. El punto también representa caracteres de control y saltos de línea. Los conjuntos negativos se interpretan como el complemento de los símbolos indicados dentro de ese universo. La notación ε representa una transición que no consume entrada, por lo que no es un carácter que el usuario deba escribir en la cadena.

Una expresión vacía o una alternativa sin contenido se considera un error de sintaxis: para representar la cadena vacía debe escribirse ε de forma explícita. El guion solo representa un rango dentro de los conjuntos y los límites deben corresponder a caracteres individuales. El alfabeto se construye según el orden en que aparecen las etiquetas de la expresión.

Construcción del AFND mediante Thompson

Después de analizar la expresión regular se recorre el árbol de sintaxis en postorden. Cada átomo produce un fragmento con un estado de entrada y uno de salida; los fragmentos se combinan según la operación correspondiente. La estructura conserva conjuntos de destinos, por lo que un estado del AFND puede tener más de una transición con la misma etiqueta.

Elemento	Regla de construcción
Literal o clase	Se crean dos estados conectados por una transición que consume la etiqueta.
ε	Se crean dos estados unidos por una transición sin consumo.
Concatenación rs	La salida del fragmento r se conecta con la entrada de s mediante ε.
Unión r|s	Una nueva entrada lleva a ambas ramas; las salidas llegan a un nuevo estado final.
Cerradura r*	Se agregan conexiones ε para entrar, salir, repetir u omitir r.

Numeración y cierre épsilon

Una renumeración final identifica la entrada real como q0 y asigna correlativos q1, q2 y así sucesivamente. El conjunto de estados finales se muestra de manera explícita. Si ε aparece en la expresión conserva su posición; cuando solo se genera durante Thompson se incorpora al final del alfabeto del AFND.

```
cierre(S):
    vistos = S; pendientes = S
    mientras pendientes no esté vacía:
        retirar q
        para cada destino t por ε desde q:
            si t no está en vistos: agregarlo a ambas
    devolver vistos
```

El conjunto de estados visitados evita recorrer indefinidamente los ciclos de transiciones ε. El cierre no consume símbolos y se utiliza tanto para obtener el estado inicial del AFD como para avanzar desde cualquier subconjunto.

Construcción de subconjuntos y clases ASCII

Cada estado del AFD representa un conjunto de estados del AFND. El estado inicial Q0 corresponde al cierre épsilon de {q0}. Una cola recorre únicamente los subconjuntos alcanzables y les asigna nombres correlativos en el orden en que aparecen.

```
S0 = cierre({q0}); registrar S0 como Q0
para cada subconjunto S pendiente:
    para cada clase de entrada C:
        T = cierre(mover(S, representante(C)))
        registrar T si todavía no existe
        matriz[Q(S), C] = Q(T)
    Q(S) es final si S intersecta F_AFND
```

Por qué se dividen los caracteres en clases

Una misma entrada puede activar etiquetas distintas. En a|., por ejemplo, el carácter a coincide con la transición literal y también con el comodín. Si el programa tomara una sola transición por prioridad, perdería posibles recorridos. Por esa razón, agrupa los 128 caracteres ASCII según las etiquetas que activan; cada clase resultante tiene un único comportamiento de transición.

Entrada en a|.	Etiquetas activas	Clase efectiva
a	Literal a y punto	{a}
Otro carácter ASCII	Solo punto	ASCII excepto a

La salida conserva Σ como el alfabeto original y presenta Γ como las clases disjuntas usadas en la matriz del AFD. Cuando se alcanza el conjunto vacío se incorpora un estado pozo; así, la función de transición queda completa. El AFD no contiene transiciones ε.

Correspondencia entre ambos autómatas

Tras consumir cualquier prefijo, el estado del AFD representa exactamente los estados a los que puede llegar el AFND, incluidos los movimientos ε. Por ello, el AFD acepta una cadena si y solo si el subconjunto alcanzado contiene un estado final del AFND.

Ejemplo de construcción: (a|b)*abb

La expresión (a|b)*abb reconoce las cadenas formadas por a y b que terminan en abb. En el ejemplo documentado, Thompson produce 14 estados: q0 es inicial y q13 es final. La parte repetitiva permite consumir cualquier secuencia de a y b antes de la terminación fija abb.

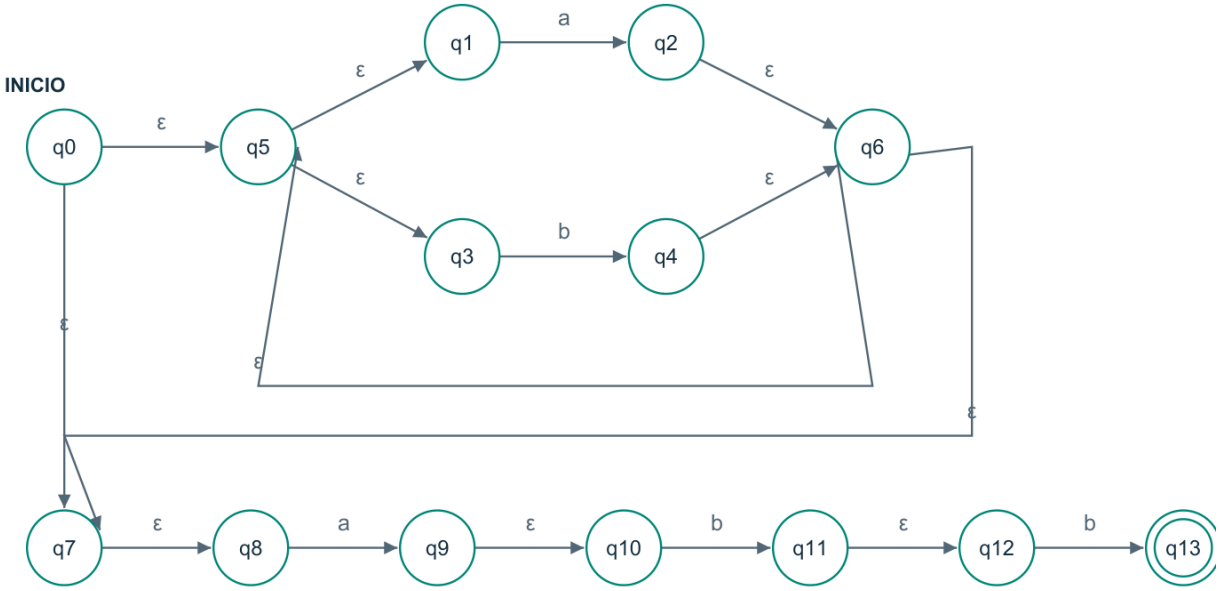


*Figura 1. AFND generado para (a|b)*abb, reproducido del reporte técnico original.*

Transiciones del AFND

```
q0|ε|{q5,q7}     q1|a|{q2}      q2|ε|{q6}
q3|b|{q4}       q4|ε|{q6}     q5|ε|{q1,q3}
q6|ε|{q5,q7}     q7|ε|{q8}     q8|a|{q9}
q9|ε|{q10}       q10|b|{q11}   q11|ε|{q12}
q12|b|{q13}
```

El retorno de q6 a q5 permite repetir la parte (a|b), mientras que la transición de q0 a q7 permite continuar directamente con el segmento abb. La matriz anterior enumera las transiciones no vacías del autómata.

Matriz del AFD y recorrido

La determinización del ejemplo genera seis estados, de Q0 a Q5. Q3 representa el estado pozo y Q5 es el estado final. La clase residual C2 comprende todos los caracteres ASCII excepto a y b; cada estado tiene una transición definida para a, b y C2.

Estado	a	b	C2: resto de ASCII
Q0	Q1	Q2	Q3
Q1	Q1	Q4	Q3
Q2	Q1	Q2	Q3
Q3 (pozo)	Q3	Q3	Q3

Estado	a	b	C2: resto de ASCII
Q4	Q1	Q5	Q3
Q5 (final)	Q1	Q2	Q3

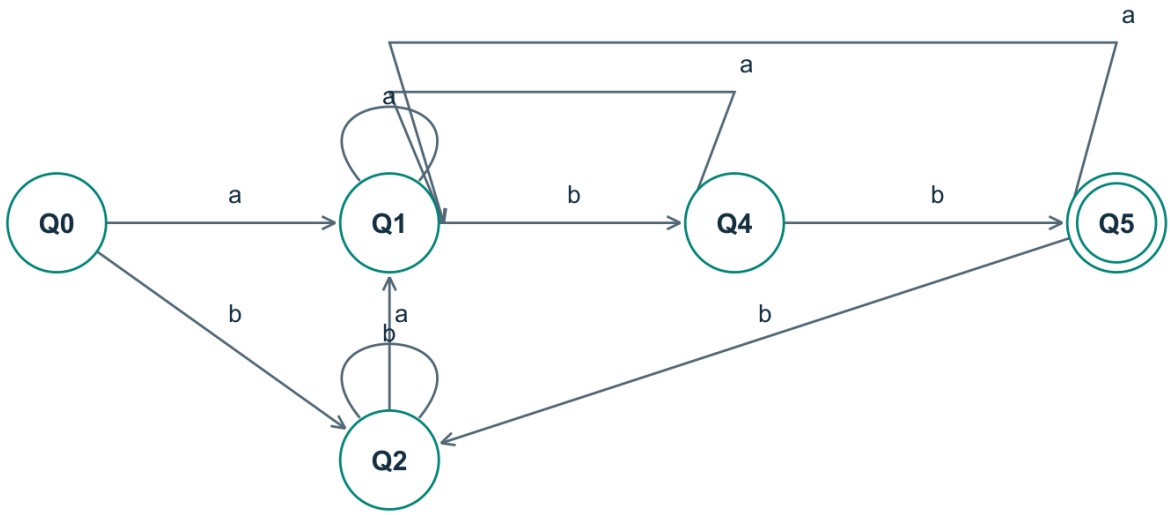


Figura 2. Diagrama parcial del AFD. La tabla anterior contiene la función completa.

Comprobación con la cadena aabb

```
Q0|a|Q1
Q1|a|Q1
Q1|b|Q4
Q4|b|Q5
Cadena aceptada
Código de retorno: 1
```

La lectura termina en Q5, que pertenece al conjunto de aceptación. El autómata no se minimiza, porque en este proyecto interesa conservar la relación entre cada estado determinista y el subconjunto del AFND que representa.

Evaluación de cadenas y tokenización

La evaluación comienza en Q0 y avanza una posición por cada símbolo de la cadena. En cada paso se consulta la clase efectiva del carácter dentro de un mapa de 128 entradas y luego se busca el siguiente estado en la matriz. La aceptación solo se decide cuando se consume toda la cadena, no al encontrar un prefijo válido.

Código	Resultado
1	Cadena aceptada.
0	Cadena rechazada.
2	Error de sintaxis, de codificación ASCII, de archivo o de límites de recursos.

Tokenización mediante er-lex

La extensión er-lex compila cada regla en un AFD y las evalúa desde cada posición del texto. Conserva la coincidencia válida más larga; en caso de empate gana la regla que aparece primero en el archivo JSON. Las reglas marcadas como skip consumen caracteres, pero no producen tokens.

```
python3 dist/er-lex --rules examples/reglas.json \
  --text 'if contador = 12 + valor;'
```

En este ejemplo se reconocen SI, IDENTIFICADOR, IGUAL, NUMERO, SUMA, IDENTIFICADOR y PUNTO_COMA. Los espacios se omiten según la regla correspondiente. El tokenizador informa línea, columna y desplazamiento al detectar caracteres sin regla, y no permite reglas que acepten la cadena vacía.

Límites y verificación

El análisis utiliza m para el tamaño de la expresión, n para la cantidad de estados del AFND, k para las clases de entrada y d para los estados alcanzables del AFD. El universo ASCII impone k menor o igual a 128.

Etapa	Costo o consideración
Parser y Thompson	El tamaño de Thompson crece con el árbol expandido; no se presupone costo lineal global del parser.
Construcción de subconjuntos	En el peor caso puede producir hasta 2^n estados; se almacenan subconjuntos y una matriz de tamaño d·k.
Simulación er-afd	Tiempo O(L) para una cadena de L caracteres.
Tokenización	Con r reglas, la exploración puede requerir $O(r \cdot L^2)$ en el peor caso.

El proyecto fija como límites preventivos 4,096 caracteres para la ER, 100 niveles de paréntesis, 20,000 estados del AFND y 4,096 estados del AFD por defecto. Este último valor puede ajustarse con --max-states. Si se supera un límite, el programa devuelve un error y no inventa un rechazo.

Resultados registrados

La ejecución local de Ubuntu completó 39 pruebas unitarias sin fallos ni errores. En los 1,499 casos con resultado esperado se obtuvieron 710 verdaderos positivos, 789 verdaderos negativos y ningún falso positivo ni falso negativo. Las 20,306 comparaciones entre el AFD y el simulador de AFND coincidieron. Además, el verificador de integración pasó sus 16 comprobaciones.

El 100 % observado corresponde exclusivamente a esas pruebas. Las comparaciones diferenciales comparten el AFND de Thompson y sirven principalmente para examinar la determinización; no demuestran por sí solas que cualquier expresión o entrada posible será interpretada correctamente.

Archivos de entrega y referencias

La matriz de cumplimiento se describe en doc/CUMPLIMIENTO.md. El paquete del proyecto incluye fuentes Python, Makefile, pruebas, ejemplos y los tres documentos PDF. Para la defensa se dispone de doc/DEFENSA.md, donde se organiza la explicación y la demostración requerida por el curso.

Los registros de la ejecución en Ubuntu se conservaron en doc/evidencias/linux/20261102T180611Z/. Esa carpeta incluye las métricas, códigos de retorno, transcripción, archivos SHA-256 y seis capturas originales. Los registros anteriores de Windows se mantienen separados.

Referencias utilizadas

1. UMG Guatemala. *ER-AFD: Convertidor de ER a AFD*. Proyecto del curso, René Ornélyz, septiembre de 2026, pp. 3–8.

2. Thompson, K. (1968). Regular expression search algorithm. *Communications of the ACM*, 11(6), 419–422. doi:10.1145/363347.363387.

3. Rabin, M. O. y Scott, D. (1959). Finite Automata and Their Decision Problems. *IBM Journal of Research and Development*, 3(2), 114–125.

4. Python Software Foundation. Documentación oficial de *zipapp* y *unittest*; consulta registrada el 30 de octubre de 2026.

El repositorio de entrega es github.com/Medec92227/er-afd.
Los resultados de CI se verifican de forma independiente.

Anexo: Revisión de la implementación

Se revisó el código de er_afd/ para contrastar la explicación del documento con la implementación ejecutada. El análisis AST y la lectura de las funciones no encontraron motores re, regex o fnmatch ni evaluación dinámica en el reconocimiento.

Etapa	Implementación revisada
Parser	Parser.parse, union, concat, repeat y atom crean el AST; escape y character_class validan las secuencias.
AFND	thompson crea estados y aristas según el AST. La numeración y las transiciones no están precargadas.
AFD	determinize procesa cierres y subconjuntos alcanzables con una cola; registra un destino por estado y clase.
Reconocimiento	DFA.step consulta la matriz y match recorre cada carácter. El lexer también usa dfa.step.

Ruta de ejecución

cli.main → compile_regex → parse_regex → thompson → determinize → DFA.match. La función compile_regex pertenece al proyecto; no llama a una biblioteca de regex.

Ejemplos construidos durante la revisión

ER	Estados AFND	Estados AFD	Transiciones AFD
a	2	3	6
ab	4	4	12
(ab|cd)*e	14	7	42

La auditoría se apoya en fuentes identificados por SHA-256 y en casos adicionales ejecutados en Ubuntu. El detalle de imports, archivos y funciones está en doc/AUDITORIA_IMPLEMENTACION.md y en resultado_adicional.json.

Esta revisión confirma la construcción automática observada y la ruta de reconocimiento tabular. Una auditoría estática y una muestra finita de pruebas no demuestran ausencia de errores para todas las entradas posibles.